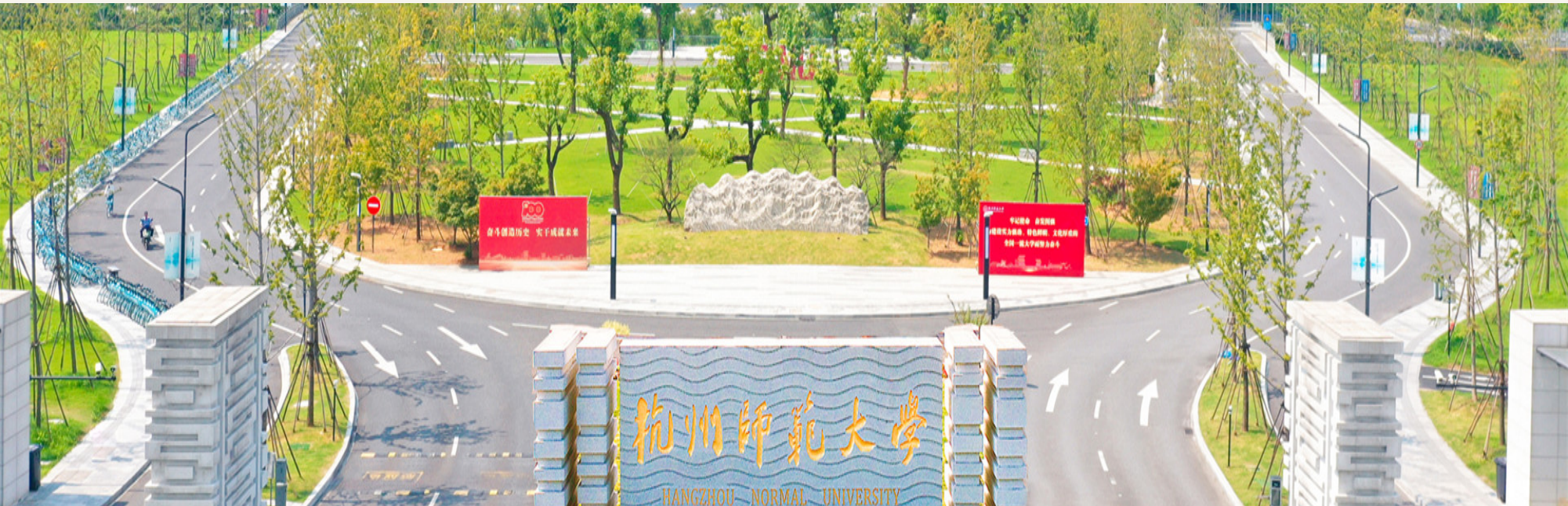
第三讲 - NLP基础算法：N-gram语言模型

张建章

阿里巴巴商学院
杭州师范大学

2026-03-01



1 语言模型

2 N-gram模型

3 N-gram模型参数估计

4 评估语言模型

5 从语言模型中采样句子

6 语言模型的泛化性

7 解决“零概率”—平滑、插值和回退

8 困惑度指标的信息论基础

9 课后编程实践：构建电商好评 N-gram 模型

语言模型（Language Model，LM）：是一种统计学习模型，其主要任务是根据已有的上下文来预测接下来的词。该模型为每个可能的下一个词分配一个概率，进而构成一个完整的概率分布；同时，语言模型也能够计算整个句子的联合概率，从而反映出不同词序排列的合理性。

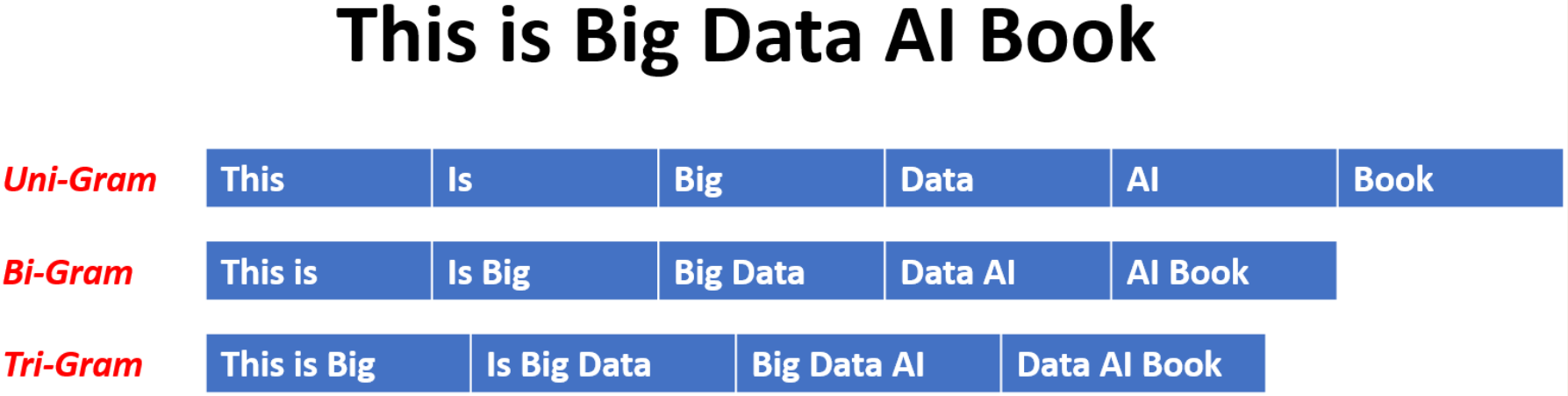
实际功能与应用场景：

- **拼写检查**：通过比较不同词序的概率，语言模型能够帮助纠正语法或拼写错误。例如，当员工在撰写面向客户的邮件时输入“The company's performance has improve”，模型会计算出 $P(\text{has improved}) > P(\text{has improve})$，从而提出语法修正建议；
- **语音识别**：在语音识别系统中，语言模型帮助区分发音相似但意义截然不同的词序。例如，在语音转文本时，模型通过概率判断得知 $P(\text{I will be back soonish}) > P(\text{I will be bassoon dish})$（“我很快就回来”的概率远大于毫无逻辑的“我将成为低音管盘子”），将语音转录为正确文本。

下一个词预测不仅是解决特定任务的手段，更是训练大语言模型的核心任务。使用海量语料训练大语言模型完成预测下一个词这个自监督任务，可以使其从海量语料中学习到丰富的语言结构与语义知识。

本讲以N-gram语言模型为例，详细讲解如何利用固定窗口内的上下文（即前n-1个词）来近似计算下一个词的概率。尽管N-gram模型本身在表达能力上较为局限，但其明确的数学形式和直观的统计方法，为理解训练集、测试集、困惑度（Perplexity）、采样、插值方法等大语言模型核心概念提供了坚实基础。

一个N-gram就是一个包含n个词的序列，n常见的取值为1 (unigram)、2 (bigram)、3 (trigram)，如下图所示：



在N-gram模型中，目标是通过已知的历史词汇（即前面的n-1个单词），预测下一个单词。具体来说，N-gram模型计算条件概率$P(w|h)$的值，其中w是下一个单词，而h是前n-1个词组成的序列。

1. 概率估计的直接方法

为计算$P(w|h)$，一种直观的方法是使用相对频率估计：统计在一个大规模语料库中，历史序列h出现的次数和h后跟w出现的次数，计算频率来估算概率。

$$P(w|h) = \frac{C(h,w)}{C(h)}$$

其中，$C(h,w)$表示序列h后出现词w的次数，C(h)表示序列h出现的总次数。

2. 链式法则（Chain Rule）

为了计算整个词序列的联合概率$P(w_1^n) = P(w_1, w_2, ..., w_n)$，可以使用链式法则，将长序列的联合概率转化为一系列条件概率的乘积：

$$P(w_1, w_2, ..., w_n) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2)...P(w_n|w_1, ..., w_{n-1}) \quad (1)$$

$$= \prod_{k=1}^{n} P(w_k|w_{1:k-1}) \quad (2)$$

这种方法理论上严谨，但在实际处理文本时会面临数据稀疏与组合爆炸的挑战。由于表达方式千变万化，特定的长词序列在训练语料库中往往从未完整出现过（即大量长序列计数为零，导致无法计算真实概率）。因此，需要引入更具落地性的近似估计方法。

马尔科夫假设

1. 马尔科夫假设

为了简化问题，N-gram模型引入了马尔科夫假设，即假设下一个单词的概率仅依赖于前N-1个单词。例如，在bigram模型中 (一阶马尔科夫假设)，预测下一个单词w的概率为：

$$P(w_n|h) \approx P(w_n|w_{n-1}) \tag{3}$$

因此，N-gram模型通过限制上下文长度，降低了计算复杂度，使模型更加高效。

2. 扩展到更高阶的N-gram

当n增大时，模型能够利用更多的上下文信息，在N-gram模型中 (N-1阶马尔科夫假设)，预测下一个单词w的概率为：

$$P(w_n|w_{1:n-1}) \approx P(w_n|w_{n-N+1:n-1}) \tag{4}$$

3. N-gram模型

将 (4)式表示的N-gram模型带入 (2)式表示的词序列联合概率，可得近似：

$$P(w_1^n) \approx \prod_{k=1}^{n} P(w_k | w_{k-N+1:k-1})$$

N=2时，应用bigram模型，词序列的联合概率可近似为：

$$P(w_1^n) \approx \prod_{k=1}^{n} P(w_k | w_{k-1})$$

最大似然估计（MLE）

N-gram模型的参数，即 $P(w_n|w_{n-N+1:n-1})$，可以通过最大似然估计（MLE）来计算。在语言模型中，MLE的直观体现就是用相对频率来估计概率 。这种方法通过获取语料库中N-gram出现的频数，并对频数进行归一化处理来得到概率估计值。具体步骤如下：

1. 频数计算

对于每一个N-gram，计算其在训练语料库中的绝对出现次数（即频数）。

2. 概率归一化

为了将频数转化为概率，需要将每个N-gram的频数除以其前置词序列的总频数。概率估计公式为：

$$P(w_n|w_{n-N+1:n-1}) = \frac{C(w_{n-N+1:n-1}w_n)}{C(w_{n-N+1:n-1})}$$

通过这种计算相对频率的方法，N-gram模型能够直接根据经验统计数据来估计词汇序列的真实发生概率。

Bigram模型参数估计示例

表 1: 2-gram计数表（局部）

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

(i, want)出现的频数为827，(i, eat)出现的频数为9。

表 2: 前置词序列计数表

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

在整个语料库中，前置词i出现的频数为2553。

表1中的第i行除以表2中的第i列，可得2-gram概率估计值，如下表。

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

课堂练习：给定如下2-gram概率值，计算句子*I want English food*的概率。

$$P(i \mid \langle s \rangle) = 0.25, P(\text{english} \mid \text{want}) = 0.0011$$

$$P(\text{food} \mid \text{english}) = 0.5, P(\langle /s \rangle \mid \text{food}) = 0.68$$

其中，<s>和</s>分别表示句首和句末。

$$\begin{aligned} P(\langle s \rangle\ i\ want\ english\ food\ \langle /s \rangle) &= P(i \mid \langle s \rangle)\, P(want \mid i)\, P(english \mid want) \\ &\quad P(food \mid english)\, P(\langle /s \rangle \mid food) \\ &= 0.25 \times 0.33 \times 0.0011 \times 0.5 \times 0.68 \\ &= 0.000031 \end{aligned}$$

关于<s>和</s>的说明

不计算 $P(\langle s\rangle)$ 的原因：$\langle s\rangle$ 不是模型需要预测的随机事件，而是模型启动预测的初始已知条件。

计算$P(\langle /s\rangle \mid food)$的原因：预测句子何时结束是一个高度不确定的变量，如果没有结束符号 $\langle /s\rangle$，模型就会失去对句子长度的控制。

在 N-gram 模型中，在句首人为添加特殊符号 $\langle s\rangle$，其唯一目的仅仅是为了给句子中的第一个真实单词提供一个上下文历史。因为在评估一句话的概率时，这句话已经存在了，所以句子开始是一个必然事件（概率为 1）。上例中，公式的第一项 $P(i \mid \langle s\rangle)$ 表达的正是："既然句子已经开始了，第一个词是 'i' 的概率是多少？"

商业类比：

假设我们在建立一个预测用户在 APP 内点击流的模型。$\langle s\rangle$ 就像是用户 '打开了 APP' 这个既定事实。我们不需要去预测用户打开 APP 的概率（因为我们的分析前提就是用户打开App使用），我们需要预测的是：已知用户打开了 APP，他第一步点击 '搜索框' 的概率是多少。这就是为什么我们直接从 $P(\text{点击搜索} \mid \text{打开APP})$ 开始计算。类似地，$\langle /s\rangle$就像是用户 '离开了 APP'，我们在预测 "用户在买了水之后，是继续买零食，还是直接去收银台结账离开（$\langle /s\rangle$）" 时，这两者的概率是完全不同的，因此$P(\text{结账离开} \mid \text{买水})$ 必须被精确计算。

N-gram模型的应用与挑战

1. 应用场景

N-gram语言模型广泛应用于各种自然语言处理任务，如拼写检查、语音识别。通过在大量文本数据上训练，N-gram模型能够有效地捕捉语言的统计特性，为后续应用提供支持。

2. 挑战

- **数据稀疏问题**：N-gram模型的一个主要问题是，当语料库中不存在某个N-gram时，模型会面临“零概率”问题，即某些词序列的概率为零。为了解决这个问题，常使用平滑技术（如Laplace平滑）来调整概率估计。此外，为避免概率乘积数值下溢，语言模型的概率均在计算和存储过程中使用对数概率。

- **参数组合的指数级爆炸**：N-gram 的参数量会随着 n 的增加呈指数级暴增，其可能组合数为 V^n（V 为词表总数）。例如，假设一个中等规模的词表 $V \approx 30,000$，2-gram（Bigram）的参数量约为 $30,000^2 = 9$亿；而如果要训练 4-gram，参数量将飙升至 $V^4 \approx 7 \times 10^{17}$ 的天文数字。由于极高的存储与计算壁垒，传统 N-gram 的 n 通常设定为 2 到 3。

评估语言模型的基本目标

1. 评价目标

评估语言模型的最终目的是判断其对未知数据的预测能力，即模型在面对未见过的测试数据时，能否较好地“拟合”数据，较优的模型会为测试数据分配较高的概率，从而体现出更好的泛化性能（学到了丰富的语言统计特性）。

2. 内在与外在评价

外在评价（Extrinsic Evaluation）：将语言模型嵌入到实际应用中，评估其在具体任务中的表现。这种评价方式直接反映了模型在特定应用场景中的有效性，但通常需要高昂的计算成本和大量的实际应用数据。

例如，有两个不同的语言模型，一个使用的是bigram模型，另一个使用的是trigram模型。在语音识别任务中，将这两个模型分别应用到语音转录系统中，比较其转录准确率。

内在评价（Intrinsic Evaluation）：指在不依赖于具体应用的情况下，通过一些统计指标直接评估语言模型本身的性能。这类评价通常使用一些标准的评估指标，如困惑度（Perplexity）等。

训练集、测试集、开发集

训练集（Training Set）：用于学习模型参数的语料库。例如，在N-gram语言模型中，使用最大似然估计，利用训练集中的N-gram计数，经过归一化处理得到模型的参数估计值。（练习题）

测试集（Test Set）：一组与训练集不重叠的、独立的语料，用于评估模型在新数据上的泛化能力。如果一个模型在训练集上表现极佳，但在测试集上效果很差，则说明模型可能过拟合训练数据，无法适用于未知数据。（真题）

开发集（Dev Set）：模型在训练过程中即使不直接将测试数据用于训练，多次反复在测试集上调试模型，也可能使模型隐式地适应测试集分布。因此，通常建议在模型开发阶段使用一个独立的开发集，待模型调试完毕后再在测试集上进行一次性评估。（模拟题）

数据集划分的原则

领域选择：测试集应能真实反映目标应用领域的语言特征。若模型用于语音识别化学讲座，其测试集应来自化学讲座文本。若构建通用语言模型，则测试集应涵盖多样化的文本来源，避免单一类型文档造成评价结果的偏倚。

数据量的权衡：理想情况下，测试集应足够大，以确保统计上的代表性和评价的稳定性，但又不能过大以至于严重影响训练数据的量。

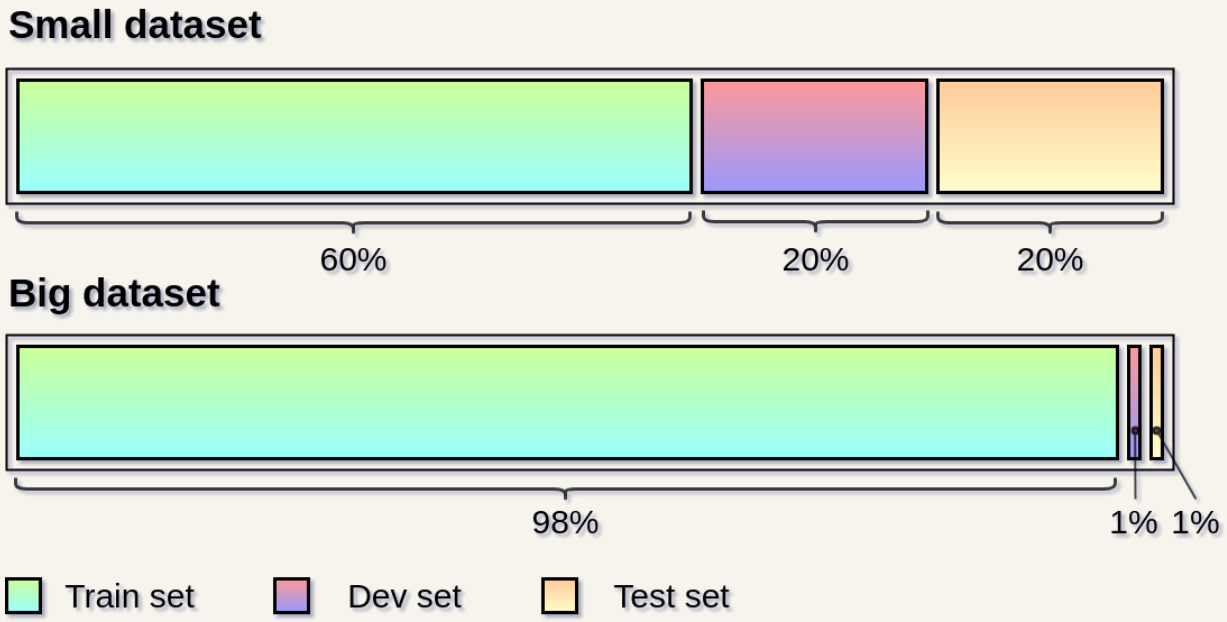


图 1: 不同规模数据集的划分比例

内部评估指标：困惑度

1. 困惑度的评价动机

语言模型的基本任务（之一）是为给定的测试文本分配概率，模型对测试集预测得越好，即为测试集赋予越高的概率，其性能就越优。然而，直接比较测试集的联合概率存在一个问题：文本越长，概率值越小，不便于跨文本比较。

为了消除文本长度对联合概率的影响，引入困惑度（Perplexity）这一评价指标，通过对测试集的联合概率进行归一化（通常是取N次根），实现了每个单词的平均预测效果度量。

对于一个包含 N 个单词的测试集 $W = w_1 w_2 \ldots w_N$，困惑度定义为：

$$\text{Perplexity}(W) = P(w_1 w_2 \ldots w_N)^{-1/N}$$

通过链式法则展开为：

$$\text{Perplexity}(W) = \left(\prod_{i=1}^{N} \frac{1}{P(w_i | w_1 \ldots w_{i-1})} \right)^{1/N}$$

上式表明，困惑度是所有条件概率倒数的几何平均值，其值与模型对测试集的预测准确性成反比。

2. 困惑度的直观解释

一个好的语言模型能够对测试集中的每个单词赋予较高的条件概率，从而使得整个测试集的联合概率较高，其倒数取N次根后的困惑度较低。在实际评价中，困惑度是衡量语言模型性能的重要指标，困惑度低表示模型对下一个单词的预测更为准确和确定。

3. 工程落地提示：多句子测试集的处理

在真实的评测中，测试集往往包含成千上万个句子。此时，将所有句子首尾相连，视为一个单一的长序列计算总概率。

- **边界处理：**利用句首 ‘<s>‘ 和句尾 ‘</s>‘ 标记跨越句子边界。
- **总词数 N 的计算：**在计算总长度 N 时，必须包含句尾标记 ‘</s>‘，但**绝对不能包含句首标记 ‘<s>‘**。因为 ‘</s>‘ 转移到 ‘<s>‘ 是概率为 1 的必然事件，包含它会引入统计偏差，人为导致困惑度失真。

困惑度在模型评价中的应用

1. 比较不同模型

模型的困惑度越低越好。困惑度计算对词汇表敏感，只有在不同模型使用相同的词汇表时，困惑度值才具有可比性。

2. 困惑度与实际任务表现

困惑度作为一种内在评价指标，不仅方便计算（无需嵌入实际应用），而且在统计上能较好地反映模型对语言序列的拟合程度。尽管困惑度通常与实际应用（如拼写检查、语音识别）的表现呈负相关，但困惑度的改善不一定能直接转化为实际任务效果的提升。

3. 数据划分要求

为确保评价结果的有效性，测试集必须与训练集严格分离。如果在模型调整过程中频繁使用测试集进行评估，可能导致隐性调参，使困惑度偏低，评价结果失真。因此，模型调整应在开发集上进行，最后在测试集上进行一次测试。

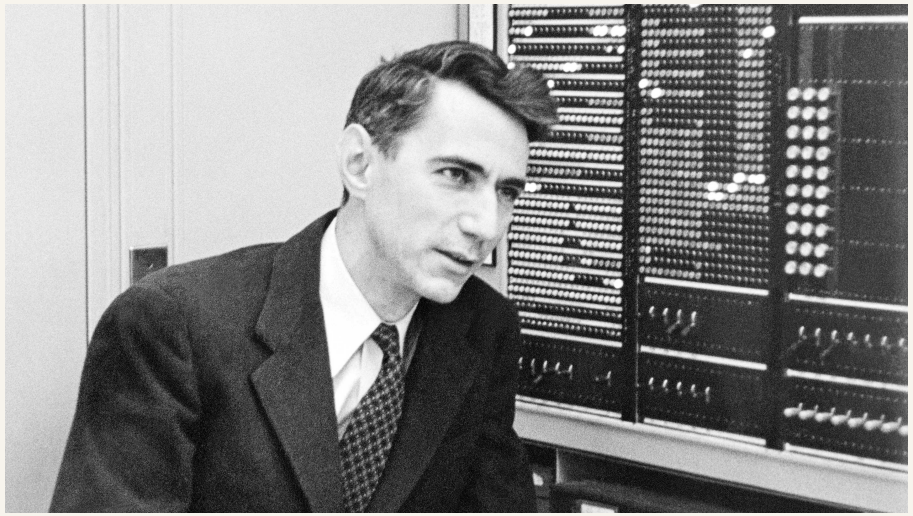
采样 (sampling)的基本思想

1. 定义

采样即根据概率分布随机选择数据点。语言模型定义了一个关于句子（或词序列）的概率分布，采样过程就是按照各个句子的概率高低随机生成句子：高概率的句子更容易被生成，低概率的则较少出现。

2. 目的

通过从模型定义的概率分布中随机抽取样本，可以“生成”出一系列句子，直观观察模型认为哪些句子更有可能出现，从而了解模型对语言结构和词序的把握情况，检测模型生成文本的连贯性和合理性。这种利用采样来可视化语言模型的方法，最早由 Shannon（1948）等人提出。纪录片《The Bit Player》



采样句子的具体过程 (N-gram语言模型生成文本的过程)

1. 以 Unigram 模型为例

- 设想词表中的词沿着区间 [0, 1] 排列，每个单词所占的区间长度与其在训练语料中的频率成正比；（以任意顺序将单词排列在数轴上，高频词对应的区间大）
- 通过随机选取 [0, 1] 内的一个数，找到其所在区间对应的词，则生成该词；（高频词对应的区间中的数字被选取到的概率更大）
- 持续这一过程，直到生成句末符号</s>，从而构成一个完整的句子。

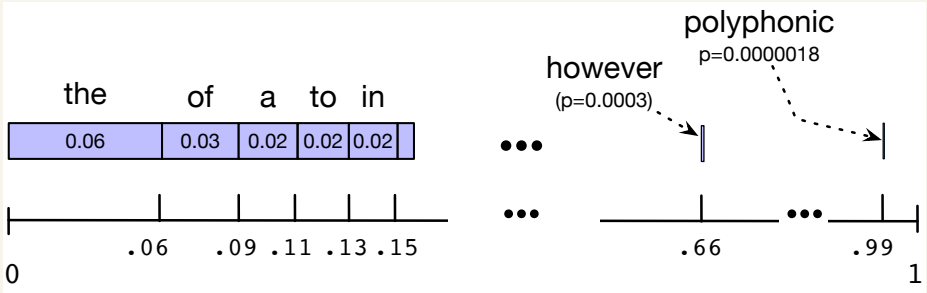


Figure 3.3 A visualization of the sampling distribution for sampling sentences by repeatedly sampling unigrams. The blue bar represents the relative frequency of each word (we've ordered them from most frequent to least frequent, but the choice of order is arbitrary). The number line shows the cumulative probabilities. If we choose a random number between 0 and 1, it will fall in an interval corresponding to some word. The expectation for the random number to fall in the larger intervals of one of the frequent words (*the*, *of*, *a*) is much higher than in the smaller interval of one of the rare words (*polyphonic*).

2. 扩展到 Bigram 模型

假设有一个词汇集合 (包括句首符号<s>和句末符号</s>)和经过训练得到的条件概率分布，且所有条件概率均满足对每个前驱词归一化，例如，对于单词"I"，$P(am|I) = 0.8$，$P(like|I) = 0.2$。

- 首先，从标记为句首的特殊符号 <s> 开始，根据 <s> 后各单词的 bigram 概率采样生成第一个词；
- 接下来，将刚生成的词作为条件，再从以该词为前缀的 bigram 分布中采样生成下一个单词；
- 重复此过程，直到生成句末符号 </s> 为止，从而形成完整的句子。

1. 泛化能力的体现

一个优秀的语言模型不仅在训练数据上拟合得好，更重要的是能在未见过的测试数据上做出准确预测，即具备较强的泛化能力。模型对测试数据赋予的概率越高，说明其预测能力越强。

2. 模型过拟合训练数据

N-gram 模型是基于训练语料统计得到的概率分布，其参数直接来源于语料中的 N-gram 计数。因此，模型不仅捕捉到一般的语言规律，还不可避免地编码了训练语料中出现的具体事实和特定模式。（I am是一般语言规律，text mining就是具体事实）

随着 N-gram 阶数的增高，即n变大，模型能利用更长的上下文捕捉细节，记住更多具体的词序信息，从而在训练集上表现出更高的预测准确性，但同时也容易陷入过拟合，即模型过分依赖于训练语料中的具体模式，导致在测试集或新数据上表现不佳。

低阶模型（如 unigram、bigram）：生成的句子往往缺乏连贯性，无法捕捉长期依赖；

高阶模型（如 trigram、4-gram）：生成的句子在局部结构上较为连贯，但当模型阶数过高时，生成的句子可能直接“复制”训练语料中的片段，显示出明显的过拟合现象。

3. 缓解过拟合

为了提高泛化能力，应选择与实际应用场景相似的训练语料，并注意数据的多样性。例如，构建用于社交媒体文本处理的语言模型时，则应采集多个不同社交媒体平台的数据，如小红书、微博、公众号、知乎、各大短视频平台等。

除了选择合适的语料外，还应严格保证测试集与训练集不重叠，防止模型“见过”测试数据，从而造成过拟合现象的低估。

“零概率”问题

由于训练语料有限，某些在测试集中可能合理出现的 N-gram 在训练集中未曾出现，其 MLE 估计为零。若某个词在特定上下文中概率为 0，则整个句子的概率也为 0，导致困惑度等评价指标失效（概率连乘）。

场景举例：假设我们正在训练一个智能客服回复模型（使用 Bigram 二元模型）。

训练集（历史对话）：包含大量词汇，比如“申请”、“退款”、“极速”、“发货”。由于历史数据的偶然性，模型恰好见过“申请退款”和“极速发货”，但从来没有见过“极速退款”这个词组。

测试集（新用户的真实提问）：用户输入了一句话 $W =$ “我要求极速退款”。

数学灾难的发生：当模型尝试计算这句真实提问的联合概率时，根据链式法则：

$$P(\text{我要求极速退款}) = P(\text{我}) \times P(\text{要求} \mid \text{我}) \times P(\text{极速} \mid \text{要求}) \times P(\text{退款} \mid \text{极速})$$

假设前三项的概率分别是 0.1、0.05、0.02。但是，因为“极速退款”在训练集中从未连在一起出现过，它的绝对频数为 0，导致 $P(\text{退款} \mid \text{极速}) = 0$ 。

$$P(\text{总句子}) = 0.1 \times 0.05 \times 0.02 \times 0 = 0$$

仅仅因为一个未见过的词组，整句合乎逻辑的商业诉求就被模型判定为“绝对不可能发生（概率为0）”。

为应对训练数据中“零概率”问题而设计的3种常用技术：

- **平滑**：通过对计数进行加常数处理（如加一或加k），为未见事件赋予非零概率，但可能过度平滑；
- **插值**：结合不同阶数 N-gram 的信息，利用低阶模型补充高阶模型的不足，并通过开发集来学习合适的权重分布；
- **回退**：当高阶 N-gram 无法获得可靠统计时，直接回退到低阶模型，并乘以固定的衰减因子，以实现一种简单高效的概率估计。

这些方法各有优劣，但都旨在改善模型对未见事件的估计，从而提高语言模型的整体泛化能力。

平滑（Smoothing）

1. 基本思想

对计数进行调整，将一部分概率质量从已见事件转移到未见事件上。

Laplace（加一）平滑：对每个 N-gram 的计数加一，即将零计数变为1，已见计数由 c 变为 $c+1$。

对于 unigram：

$$P_{\text{Laplace}}(w_i) = \frac{c(w_i)+1}{N+V}$$

对于 bigram：

$$P_{\text{Laplace}}(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)+1}{C(w_{n-1})+V}$$

其中 V 为词汇表大小。

局限性：此方法简单直观，但往往会过度转移概率质量，导致已见事件的概率被显著降低，从而影响模型性能。

Add-k 平滑：与加一平滑类似，但不是加 1，而是加一个较小的常数 k（例如0.5或0.01），以减少对已见计数的过度修正。

$$P^*_{\text{Add-k}}(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n) + k}{C(w_{n-1}) + kV}$$

调优要求：需要利用开发集 (Dev Set)来确定一个合适的 k 值。

为什么分母要加 V（词汇表大小）?

为了保证所有概率之和依然等于 1

对于 Unigram（一元模型）：假设我们的词汇表里共有 V 个不重复的单词。在 Laplace 平滑中，我们给每一个单词的出现频数都强行加上了 1 。既然有 V 个单词，意味着我们总共“无中生有”地派发了 V 个频数。为了保证计算出来的所有单词概率加起来总和依然是 1（即概率必须被正确归一化），分母（总词数 N）就必须相应地加上这额外多出来的 V 次观察值。

对于 Bigram（二元模型）：在计算条件概率 $P(w_n|w_{n-1})$ 时，分母 $C(w_{n-1})$ 实际上代表的是“所有以 w_{n-1} 开头的二元词组的总频数”。因为跟在 w_{n-1} 后面的下一个词有 V 种可能（也就是词汇表中任何一个词），当我们给这 V 种组合的频数各自都加了 1 之后，以 w_{n-1} 开头的总频数自然就增加了 V 。

插值（Interpolation）技术

1. 基本思想

当高阶 N-gram 的计数不足时，可以利用低阶 N-gram（如 unigram、bigram）的信息对其进行补充。插值方法通过线性组合不同阶数 N-gram 的概率，平滑地整合上下文信息，从而提高对未见事件的估计。

简单线性插值：对于 trigram 模型，利用 unigram、bigram 和 trigram 的概率进行组合，其形式为：

$$\hat{P}(w_n|w_{n-2}w_{n-1}) = \lambda_1 P(w_n) + \lambda_2 P(w_n|w_{n-1}) + \lambda_3 P(w_n|w_{n-2}w_{n-1})$$

其中 $\lambda_1 + \lambda_2 + \lambda_3 = 1$。

上下文条件插值：为使插值更灵活，可令各个 λ 权重依赖于上下文，即：

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) = &\lambda_1(w_{n-2}, w_{n-1})P(w_n) \\ &+ \lambda_2(w_{n-2}, w_{n-1})P(w_n|w_{n-1}) \\ &+ \lambda_3(w_{n-2}, w_{n-1})P(w_n|w_{n-2}w_{n-1})\end{aligned}$$

简单插值和上下文插值的权重λ通过在开发集上最大化似然函数来学习，使更可靠的 N-gram 拥有更高的权重。

回退（Backoff）技术

1. 基本思想

当高阶 N-gram (例如 trigram)无法获得可靠统计时 (例如计数为零)，直接回退到低阶模型 (例如 bigram)，并乘以固定的衰减因子，以实现一种简单高效的概率估计。

2. Stupid Backoff 算法

如果某个 N-gram 的计数大于零，则使用其 MLE 估计；否则，将概率设置为较**低阶模型概率**乘以一个固定的衰减因子 λ (例如 0.4)，递归回退至 unigram。

$$S(w_i \mid w_{i-N+1:i-1}) = \begin{cases} \dfrac{\text{count}(w_{i-N+1:i})}{\text{count}(w_{i-N+1:i-1})}, & \text{if count}(w_{i-N+1:i}) > 0; \\ \lambda\, S(w_i \mid w_{i-N+2:i-1}), & \text{otherwise.} \end{cases}$$

Stupid Backoff 不试图形成一个完整的概率分布，其主要目的是作为一个简单、计算高效的启发式方法来处理稀疏性问题，λ通常为0.4。

熵的定义及其在语言中的应用

1. 熵的定义

给定随机变量 X 及其概率分布 $p(x)$，熵定义为

$$H(X) = -\sum_{x \in X} p(x) \log_2 p(x)$$

熵衡量了随机变量的不确定性，直观上可以看作编码某一信息所需的最少平均比特数 (bits)。

- 当 X 表示单个词时，熵反映了对单词选择的不确定性；

- 当 X 表示一个词序列 $W = w_1, w_2, \ldots, w_n$ 时，熵反映了整个语言的统计特性。为了得到每个词的平均不确定性，可以定义熵率，即：

$$\frac{1}{n} H(w_1, w_2, \ldots, w_n)$$

对于语言L，需要考虑无限长序列，即$n \to \infty$，将语言L视为一个生成词序列的随机过程，则语言的熵率可以定义为：

$$
\begin{aligned}
H(L) &= \lim_{n\to\infty} \frac{1}{n} H(w_1, w_2, \ldots, w_n) \\
&= -\lim_{n\to\infty} \frac{1}{n} \sum_{w_{1:n}\in L} p(w_1, w_2, \ldots, w_n) \log p(w_1, w_2, \ldots, w_n)
\end{aligned}
$$

2. Shannon-McMillan-Breiman 定理

如果一个随机过程（如语言L）满足一定的正则性条件，即，既平稳又遍历，则有

$$
H(L) = \lim_{n\to\infty} -\frac{1}{n} \log p(w_1, w_2, \ldots, w_n)
$$

这一定理说明，一个足够长的文本样本其平均对数概率可以近似于语言的真实熵率，从而为利用单个长序列估计语言（模型）的熵提供了理论依据。

交叉熵与模型评价

1. 交叉熵的定义

当不知道真实分布 p 时，可以用模型 m 来近似。交叉熵（率）定义为：

$$H(p, m) = \lim_{n\to\infty} -\frac{1}{n} \sum_{w_{1:n}\in L} p(w_1, \ldots, w_n) \log m(w_1, \ldots, w_n)$$

应用Shannon-McMillan-Breiman 定理，对于满足正则条件的随机过程，可用单个长序列来近似估计交叉熵（率）：

$$H(p, m) = \lim_{n\to\infty} -\frac{1}{n} \log m(w_1 w_2 \ldots w_n)$$

2. 交叉熵与真实熵的关系

交叉熵 $H(p, m)$ 总是不低于真实熵 $H(p)$（即 $H(p) \leq H(p, m)$ ），且两者之间的差值反映了模型 m 的准确程度。因此，在模型比较中，交叉熵较低的模型通常被认为对数据分布的近似更为准确。

困惑度与交叉熵的关系

使用一个固定长度 (N)的足够长的序列 W，即长度为N的测试集，来近似计算模型与真实分布之间的交叉熵，则交叉熵（率）的估计表达式可以写为：

$$H(W) = -\frac{1}{N}\log P(w_1 w_2 \ldots w_N)$$

困惑度是评价语言模型对测试集预测能力的指标，其定义为测试集联合概率的归一化倒数：

$$\text{Perplexity}(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$

因此，困惑度可以表示为：

$$\text{Perplexity}(W) = 2^{H(W)}$$

困惑度与交叉熵之间的关系：模型对测试集赋予的平均对数概率越高，交叉熵就越低，所训练出的语言模型越接近真实分析，相应地，困惑度就越低，说明该模型的预测能力越强。

本小节的核心内容在于建立了困惑度与熵、交叉熵（率）之间的理论联系，为使用困惑度作为评价语言模型性能的指标提供了信息论基础。具体地：

- 通过定义熵与熵率，说明了信息的不确定性；

-根据Shannon-McMillan-Breiman 定理，用长序列的平均对数概率估计语言（模型）的熵率；

- 定义了交叉熵（率），并指出交叉熵（率）是对真实熵（率）的上界，其差值反映了模型的精度；

- 建立交叉熵（率）与困惑度的联系，模型对测试集赋予的平均对数概率越高，交叉熵（率）越低，语言模型越接近真实语言分布，相应地，困惑度越小，即：

$$\text{Perplexity}(W) = 2^{H(W)}$$

这一系列理论构建解释了困惑度为何采用测试集概率的归一化倒数，因为要建立与交叉熵的关系，而交叉熵又是衡量模型准确程度的信息论标准。因此，困惑度定量反映模型对语言的预测能力，是衡量语言模型好坏的重要内在指标。

1. 实验数据与工具

- **评论数据集（任选下面一个数据集，提取好评）：**
  - 大众点评评论数据集；
  - 京东评论数据集；
  - 自己寻找的其他公开可用的评论数据集；
- **工具库：** 使用 Python 的 `nltk.lm`模块（借助大模型自学该模块用法）。

2. 实验步骤与要求

- **数据划分：** 将选定的好评数据按 8:2 比例划分为训练集与测试集。
- **预处理：** 无需分词，直接将每个中文字符（字）作为一个独立的词（Token）。
- **模型训练：** 从 $N \in \{1, 2, 3, 4\}$ 中任选两个不同的 N 值，利用训练集训练这两个 N-gram 模型。
- **定量评估：** 在测试集上分别计算这两个模型的困惑度，对比它们对未见数据的预测能力。
- **定性评估：** 利用这两个模型，分别采样生成 5-10 句虚拟好评。观察并总结不同阶数模型生成的评论在连贯性和重复性上的特点。

THE END